

Relatório de Auditoria de Segurança — ACAD Protocol MVP

Projeto: ACAD Protocol

Aluno: David Damasceno da Frota

Data: Abril/2026

Versão Solidity: ^0.8.20

Ferramentas: Análise manual + Slither + Mythril + Hardhat

1. Escopo da Auditoria

Contrato	Arquivo
AcadToken	contracts/ERC20Token.sol
AcademicCertificate	contracts/NFT.sol
AcadStaking	contracts/Staking.sol
AcadGovernance	contracts/Governance.sol

2. Como Executar as Ferramentas

Slither (análise estática)

```
# Instalar
pip install slither-analyzer

# Analisar todos os contratos
slither . --solc-remaps "@openzeppelin/=node_modules/@openzeppelin/
@chainlink/=node_modules/@chainlink/"

# Relatório detalhado
slither . --print human-summary
```

Mythril (análise simbólica)

```
# Instalar
pip install mythril

# Analisar contrato de staking (maior superfície de ataque)
myth analyze contracts/Staking.sol \
  --solc-json mythril-config.json \
  --execution-timeout 120
```

```
# Analisar governança  
myth analyze contracts/Governance.sol
```

Hardhat Coverage

```
npx hardhat coverage
```

3. Achados por Contrato

3.1 AcadToken.sol — Nenhum achado crítico

ID	Severidade	Descrição	Status
T-01	Info	Supply inicial fixo em 1M — sem cap máximo	Aceito por design
T-02	Info	MINTER_ROLE pode ser concedido a múltiplos endereços	Aceito por design

Mitigações aplicadas:

- `require(to != address(0))` no `mint()` — previne queima acidental.
- `AccessControl` ao invés de `Ownable` simples — controle granular por papel.
- Solidity 0.8.20 — overflow/underflow revertidos automaticamente.

3.2 AcademicCertificate.sol — Nenhum achado crítico

ID	Severidade	Descrição	Status
N-01	Info	Sem mecanismo de revogação de certificado	Aceito por design
N-02	Info	Metadados dependem de disponibilidade do IPFS	Limitação externa

Mitigações aplicadas:

- `require(bytes(uri).length > 0)` — impede NFT sem metadados.
- `_safeMint` ao invés de `_mint` — verifica se destinatário aceita NFTs.

3.3 AcadStaking.sol — Achados e mitigações

ID	Severidade	Descrição	Status
S-01	Alto	Risco de reentrância em funções de transferência	Mitigado
S-02	Médio	Oracle staleness — dado do Chainlink pode estar desatualizado	Parcialmente mitigado

ID	Severidade	Descrição	Status
S-03	Baixo	<code>rewardPoolBalance</code> pode ser insuficiente para recompensas acumuladas	Mitigado com <code>require</code>
S-04	Info	<code>updateRewardRate()</code> pode ser chamado por qualquer um	Aceito por design (pública)

Mitigações aplicadas:

S-01 — Reentrância:

```
// Padrão Checks-Effects-Interactions aplicado
stakedBalance[msg.sender] -= amount; // Effect ANTES
totalStaked -= amount;                // Effect ANTES
stakingToken.transfer(msg.sender, amount); // Interaction DEPOIS
// + modifier nonReentrant em todas as funções de estado
```

S-02 — Oracle Staleness:

```
// Recomendação (não implementada na versão básica):
// (, int256 price,, uint256 updatedAt,) = priceFeed.latestRoundData();
// require(block.timestamp - updatedAt < 1 hours, "Oracle data stale");
```

Nota: Para produção, adicionar verificação de `updatedAt` para garantir que o dado não está desatualizado.

S-03 — Pool insuficiente:

```
require(rewardPoolBalance >= reward, "Pool de recompensas insuficiente");
```

3.4 AcadGovernance.sol — Achados e mitigações

ID	Severidade	Descrição	Status
G-01	Médio	Flash loan attack — poder de voto baseado em saldo spot	Aceito (escopo MVP)
G-02	Baixo	Sem timelock — proposta executada imediatamente após prazo	Aceito (escopo MVP)
G-03	Info	Sem quórum mínimo de votos NÃO (só total de votos)	Aceito por design

G-01 — Flash Loan:

Em produção, usar snapshot de saldo no momento da criação da proposta (ex: `ERC20Votes` do OpenZeppelin com `getPastVotes`). No MVP simplificado, o saldo spot é suficiente para demonstrar o conceito.

Mitigações aplicadas:

- `require(hasVoted[proposalId][msg.sender])` — impede voto duplo.
- `require(!proposal.executed)` — impede execução dupla.
- `require(block.timestamp > proposal.deadline)` — garante período de votação respeitado.
- Quórum mínimo de 1.000 ACAD.

4. Resumo Executivo

Severidade	Encontrados	Mitigados	Aceitos por Design
Crítico	0	—	—
Alto	1	1 (S-01)	0
Médio	2	1 (S-03)	1 (G-01)
Baixo	2	1 (S-03)	1 (G-02)
Info	4	—	4

Conclusão: Nenhuma vulnerabilidade crítica identificada. Os principais riscos de segurança (reentrância, controle de acesso, validações de entrada) foram mitigados. Os achados de severidade médio aceitos por design são limitações conhecidas de MVPs simplificados, documentadas para melhoria futura.

5. Recomendações para Versão de Produção

1. **Oracle:** Adicionar verificação de `updatedAt` no Chainlink para evitar dados stale.
2. **Governança:** Usar `ERC20Votes` com `getPastVotes` para snapshot de poder de voto.
3. **Timelock:** Adicionar período de espera entre aprovação e execução de propostas.
4. **Staking:** Adicionar cap de supply de recompensas por período.
5. **Auditoria profissional:** Contratar uma auditoria completa (Trail of Bits, OpenZeppelin, etc.) antes do deploy em mainnet.

6. Cobertura de Testes — Resultados Reais

Executado em: 27/04/2026 | **Hardhat** + Solidity 0.8.25 | EVM: cancun

```
npx hardhat test
```

```
ACAD Protocol — Suite de Testes Completa
```

```
AcadToken — ERC-20
```

- ✓ deve ter nome, símbolo e decimais corretos
- ✓ supply inicial deve ser 1.000.000 ACAD

- ✓ owner pode mintar novos tokens com MINTER_ROLE
- ✓ deve rever se endereço sem MINTER_ROLE tentar mintar
- ✓ usuário pode queimar (burn) seus próprios tokens
- ✓ deve rever mint para zero address

AcademicCertificate – ERC-721

- ✓ deve mintar NFT e atribuir ao destinatário correto
- ✓ deve armazenar a URI corretamente
- ✓ deve emitir o evento CertificateMinted com argumentos corretos
- ✓ tokenId deve incrementar a cada mint
- ✓ deve rever se endereço sem MINTER_ROLE tentar mintar NFT
- ✓ deve rever mint com URI vazia

AcadStaking – Staking com Chainlink

- ✓ usuário pode fazer stake e saldo é atualizado
- ✓ deve emitir evento Staked
- ✓ usuário pode fazer unstake e receber tokens de volta
- ✓ recompensas devem acumular com o tempo
- ✓ deve rever stake de valor zero
- ✓ deve rever unstake com saldo insuficiente
- ✓ deve rever unstake de valor zero
- ✓ updateRewardRate: ETH > \$3.000 → taxa sobe 50%
- ✓ updateRewardRate: ETH < \$1.500 → taxa cai 30%
- ✓ updateRewardRate: ETH entre \$1.500–\$3.000 → taxa base
- ✓ deve emitir evento RewardRateUpdated
- ✓ emergencyWithdraw retorna tokens sem recompensas
- ✓ emergencyWithdraw deve rever se não há tokens em staking

AcadGovernance – DAO

- ✓ usuário com ≥ 100 ACAD pode criar proposta
- ✓ deve rever criação de proposta com < 100 ACAD
- ✓ deve rever criação de proposta com descrição vazia
- ✓ proposalCount incrementa a cada proposta
- ✓ usuário pode votar SIM em proposta ativa
- ✓ deve rever voto duplo na mesma proposta
- ✓ deve rever voto após prazo encerrado
- ✓ pode executar proposta com quórum e prazo encerrado
- ✓ deve rever execução sem quórum mínimo (1.000 ACAD)
- ✓ deve rever execução antes do prazo
- ✓ deve rever dupla execução
- ✓ proponente pode cancelar proposta antes da execução
- ✓ deve rever voto em proposta cancelada
- ✓ getProposalStatus retorna status correto

39 passing (5s)

Resultado: 39/39 testes passando — 0 falhas.

7. Integração Web3 (ethers.js) — Resultado Real

Script: `scripts/demo-web3.js`

Executado em: 27/04/2026 | Hardhat localhost

ETAPA 1: Leitura on-chain – AcadToken (ACAD), supply 1.000.000, ethers.js OK
ETAPA 2: Transfer ERC-20 – 5.000 ACAD para aluno e membro | gas: ~51.000
ETAPA 3: Mint NFT – TokenId 1, URI IPFS, evento CertificateMinted emitido
ETAPA 4: Staking – 2.000 ACAD em staking, Chainlink ETH/USD=\$2000, updateRewardRate OK
ETAPA 5: DAO – Proposta criada (ID 2), 794.000 ACAD votos SIM, evento Voted emitido

Total em staking: 3.000 ACAD | NFTs emitidos: 2 | Propostas: 3
Integração Web3 com ethers.js: CONCLUÍDA 